# Architecture Overview

System layers and module responsibilities

## System layers

UniQOS DB is organised in four layers:

1. **Storage backends** (`sd_storage.c`, `heap_storage.c`) – read/write fixed 512 B sectors, behind a common `Storage` interface (`storage.c`) that stamps every sector with a CRC-32 trailer on write and checks it on read.
2. **Index & allocation structures** – the hash table (phone number -> sector), the sorted array (alphabetical name index), two free-list stacks (contact/message sector allocators), the usage bitmap, and two ring buffers (message history, call history).
3. **Record layer** – `contact.c` and `message.c` pack/unpack records within a sector; `chat_order.c` tracks most-recent-chat-first ordering.
4. **Database API** (`database.c`) – aggregates all of the above behind the public `database_*` functions, journalled through `journal.c` for power-fail safety.

A `Database` struct ties one instance of each structure together. The same `source/app` code runs on the STM32 target (FreeRTOS, SDMMC-backed) and on the host (pthread, heap- or file-backed) via the `platform_os_*` abstraction in `platform/`.

## Module responsibility table

| Module | Responsibility |
|---|---|
| `database.c` | Public API: contact/message/call/chat operations, aggregates every structure below |
| `hash_table.c` | Phone number -> sector index via DJB2 + double hashing; largest module, also drives startup reconstruction |
| `sorted_array.c` | RAM-only alphabetical name index (8 B keys + hash slot); chosen over an AVL tree after benchmarking – see Figure 7 |

| | |
|---|---|
| `chat_order.c` | Most-recent-chat-first ordering of hash slots, rebuilt via heapsort at boot |
| `call_history.c` / `message_history.c` | Ring-buffer-backed call log / global message log |
| `message.c` / `contact.c` | Record layout and per-sector packing for messages / contacts |
| `journal.c` | Rollback journal (`EMPTY`/`ACTIVE`/`COMMITTED`) for power-fail-safe writes |
| `superheader.c` | Sector 0 metadata: magic number, version, DB status |
| `usage_bitmap.c` | Persistent used/free sector bitmap; drives startup reconstruction |
| `free_list_stack.c` | Sector allocator (push/pop) for defragmentation – one for contacts, one for messages |
| `ring_buffer.c` | Generic SD-backed ring buffer underlying message/call history, with binary-search reconstruction on boot |
| `storage.c` + `sd_storage.c` / `heap_storage.c` | Generic sector I/O + per-sector CRC-32 trailer, pluggable backend |
| `crc.c` / `crc_stm32.c` | Software vs. hardware (STM32 CRC peripheral) CRC-32 |
| `iterator.c` | Generic next/prev/get interface shared by the usage bitmap, hash table and ring buffer |
| `db_task.c` | FreeRTOS/pthread task wrapping the database behind a request/response queue, used by the demos and `db_cli` |
| `fmc_func.c` / `lcd_font.c` / `lcd_log.c` | ST7789V LCD driver (FMC bus), generated font, scrolling log widget – app-level UI, not DB |
| `mem_layout.h` | Pure constants header: sector sizes, hash table size, on-disk region layout |

# Design parameters

| | |
|---|---|
| Hash table size | 14,293 slots |
| Load capacity | 70% before reconstruction is recommended |
| Hashing method | DJB2 on the phone number |
| Collision handling | Double hashing with tombstoning (cuckoo hashing rejected – too much RAM on the H723; see Figure 7) |
| Sector size | 512 B |
| Contact | 81 B, 6 per sector |
| Message | 164 B, 2 per sector |

| | |
|---|---|
| RAM_D1 usage | 200.092 KB / 320 KB (AXI SRAM) – hash table + sorted-array name index + both free-list stacks |

The hash table, sorted array and free-list stacks are placed in RAM_D1 via `__attribute__((section(".ram_d1")))` (see `usage_bitmap.c`, `db_main.c`, `persist_demo.c`, `mh_main.c`) because they're too large for DTCM or the default task stack. DTCM is reserved for the rest of the phone firmware's code.

# Data structures

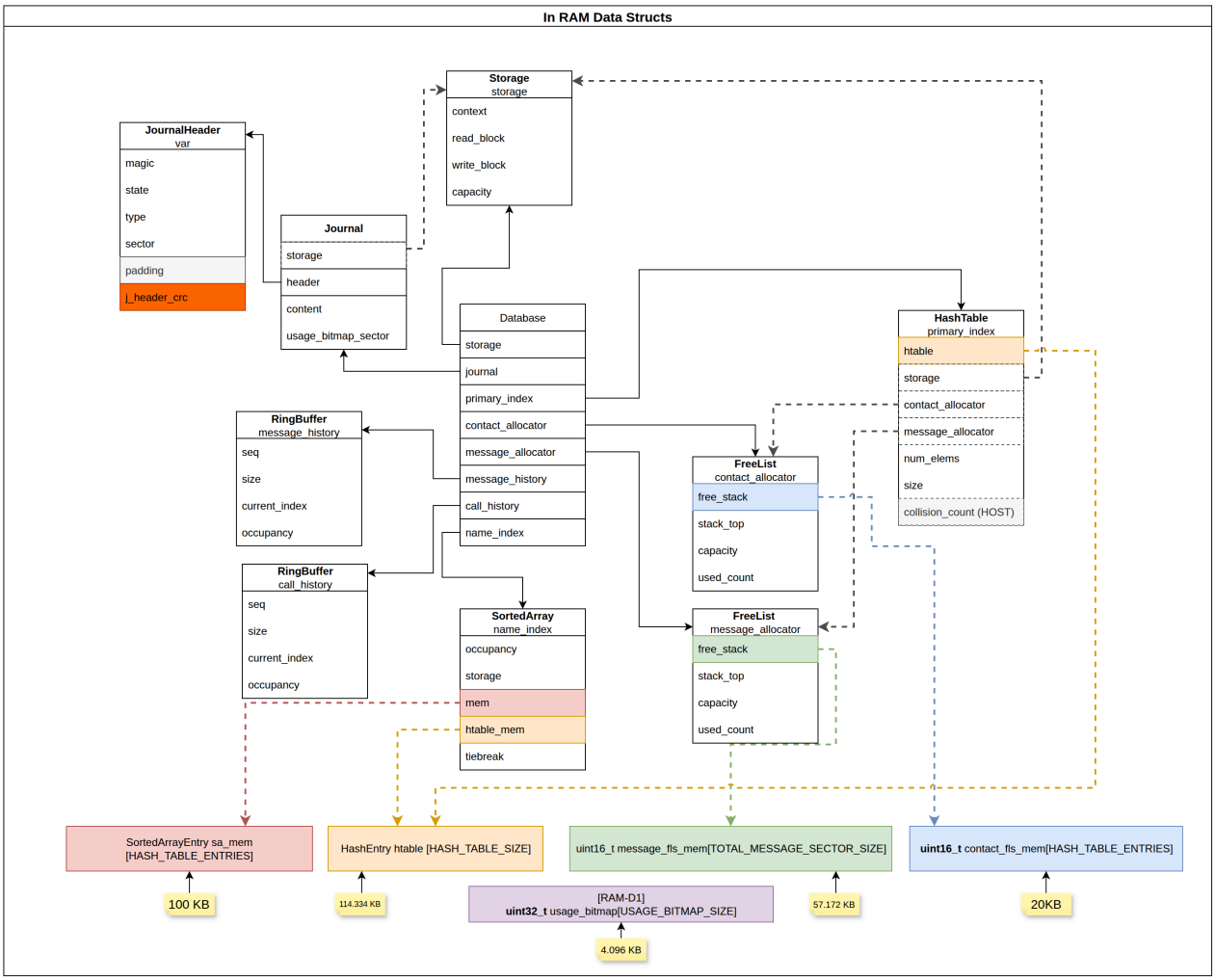



Figure 1: Every on-disk and in-RAM struct and how they reference each other, including the per-sector CRC trailer (orange) and padding (grey).

# SD card layout

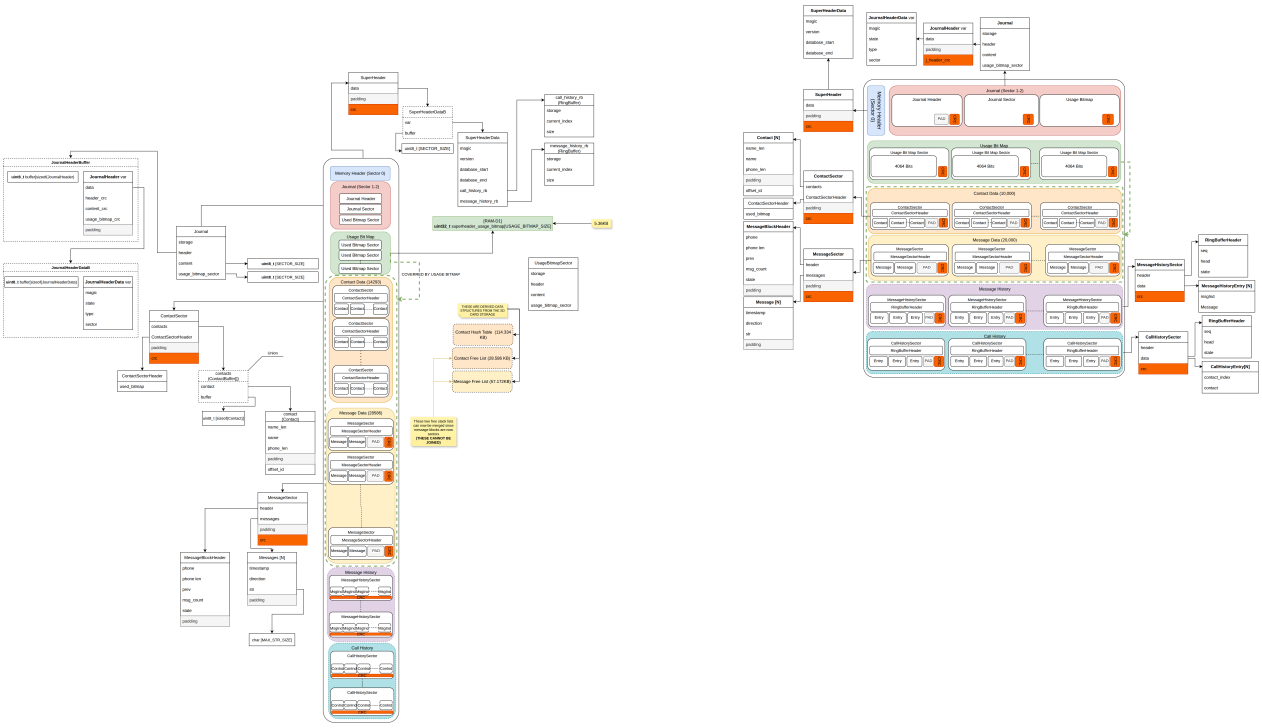


Figure 2: The SD card broken into memory blocks: superheader, journal, usage bitmap, contact data, message data, message history ring buffer, call history ring buffer. See `mem_layout.h` for the exact region macros.

# Hash table reconstruction

On every boot, the in-RAM hash table is rebuilt from the SD card by walking the usage bitmap: unused sectors go back on the free-list stack, used sectors are re-hashed from their phone number back into the table.

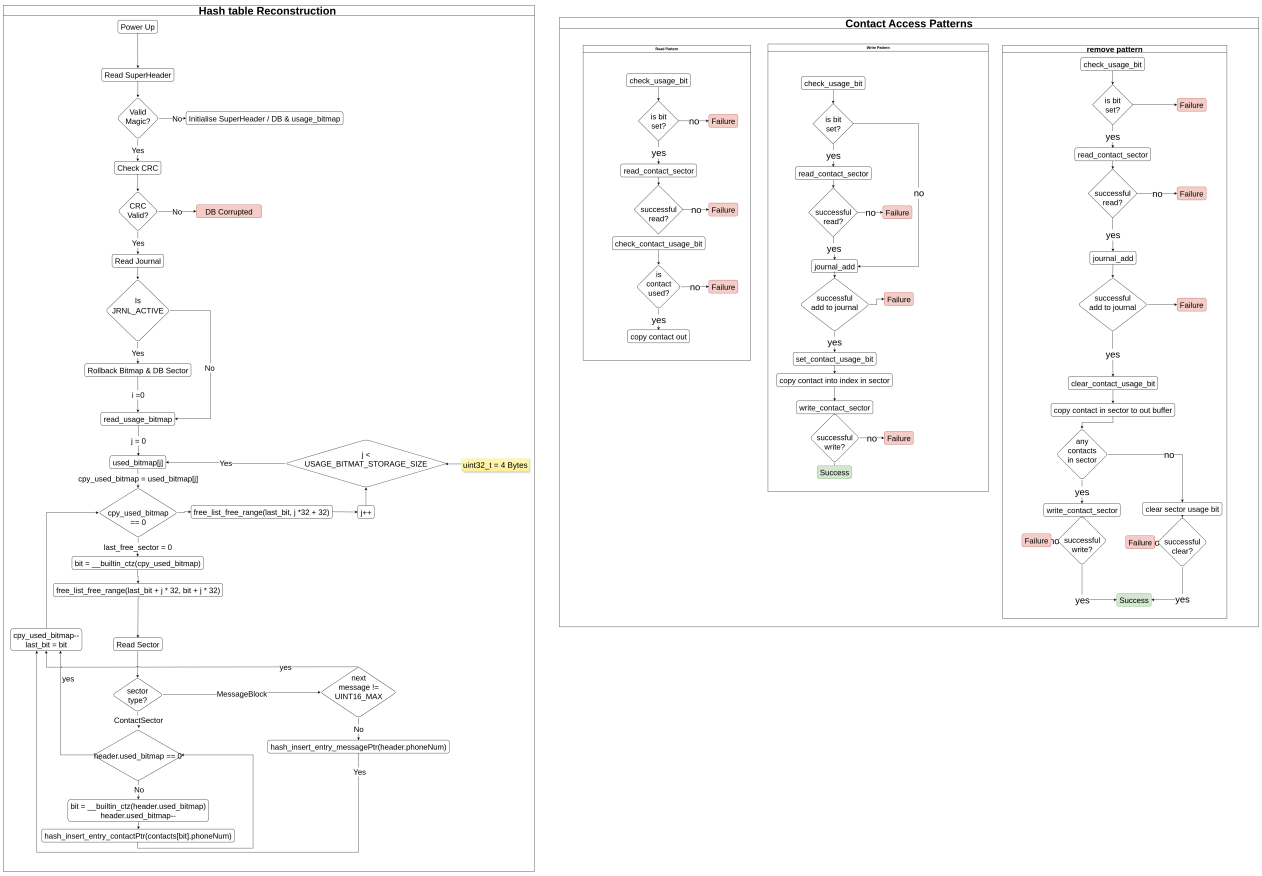



Figure 3: Reconstruction flow: usage bitmap walk, free-list repopulation, and the contact/message re-hashing paths.

# Rollback journal

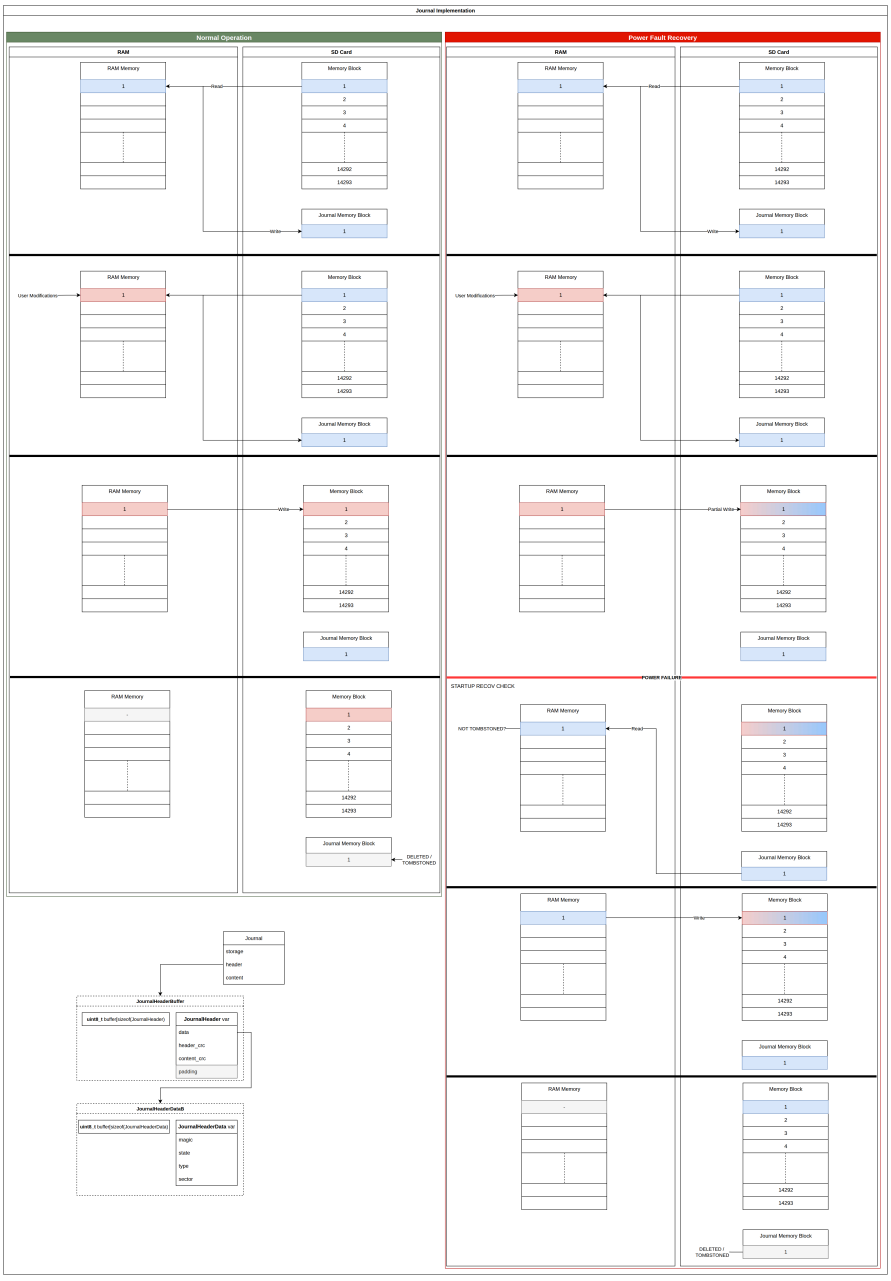






Figure 4: Journal states and the successful vs. unsuccessful read/write scenarios. A rollback happens if the journal is `ACTIVE` (not `EMPTY` or `COMMITTED`) on power-up.

# Message and contact access pattern

Contacts and messages are separate static memory blocks because their sector structures differ: a contact's index is its position **inside** a shared sector (6 contacts/sector), while a message's index **is** its sector (2 messages/sector). This is also why there are two separate free-list stacks.

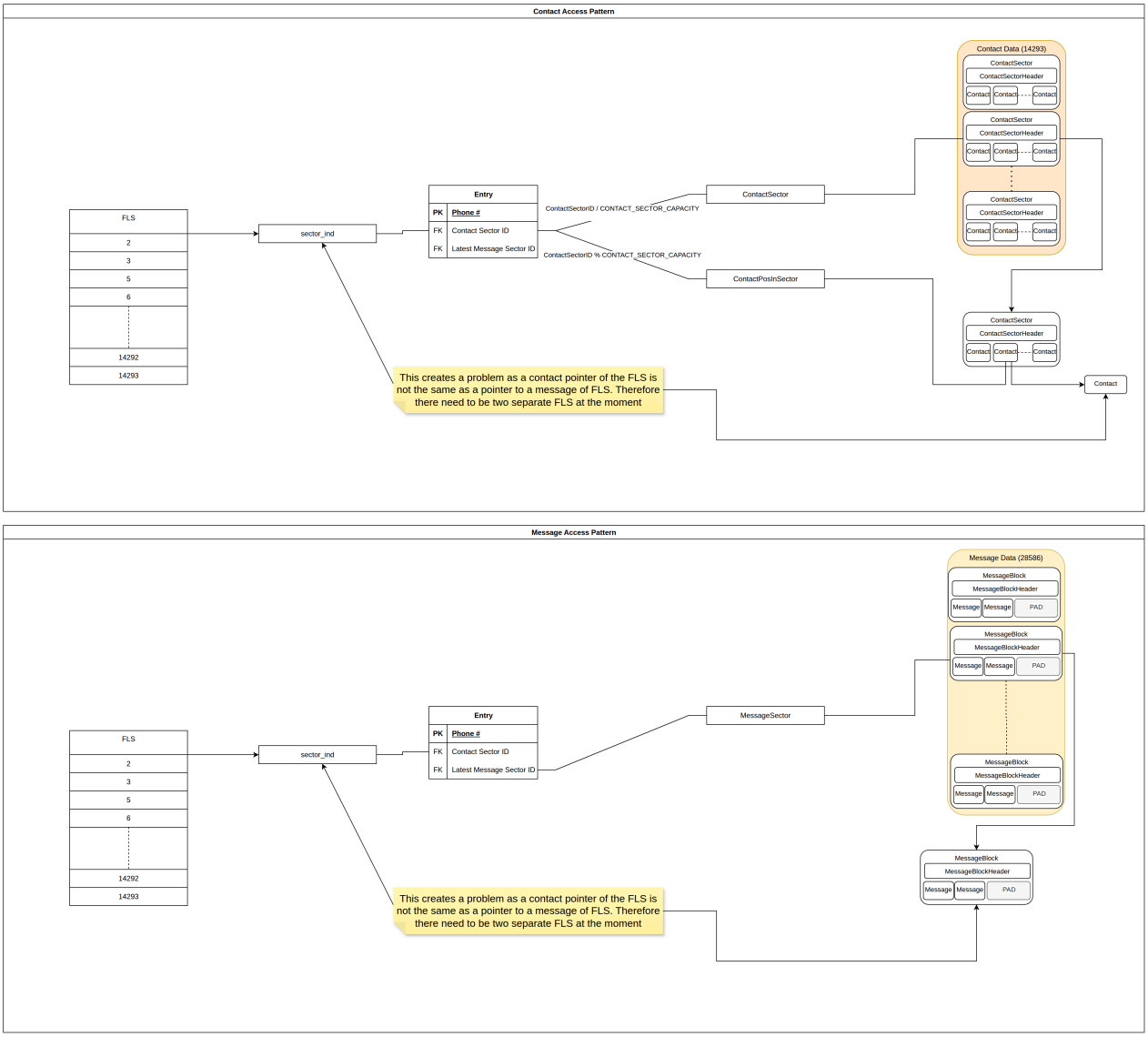



Figure 5: Indexing difference between contact sectors (packed, 6/sector) and message sectors (whole-sector addressed).

# Defragmentation (free list stack)

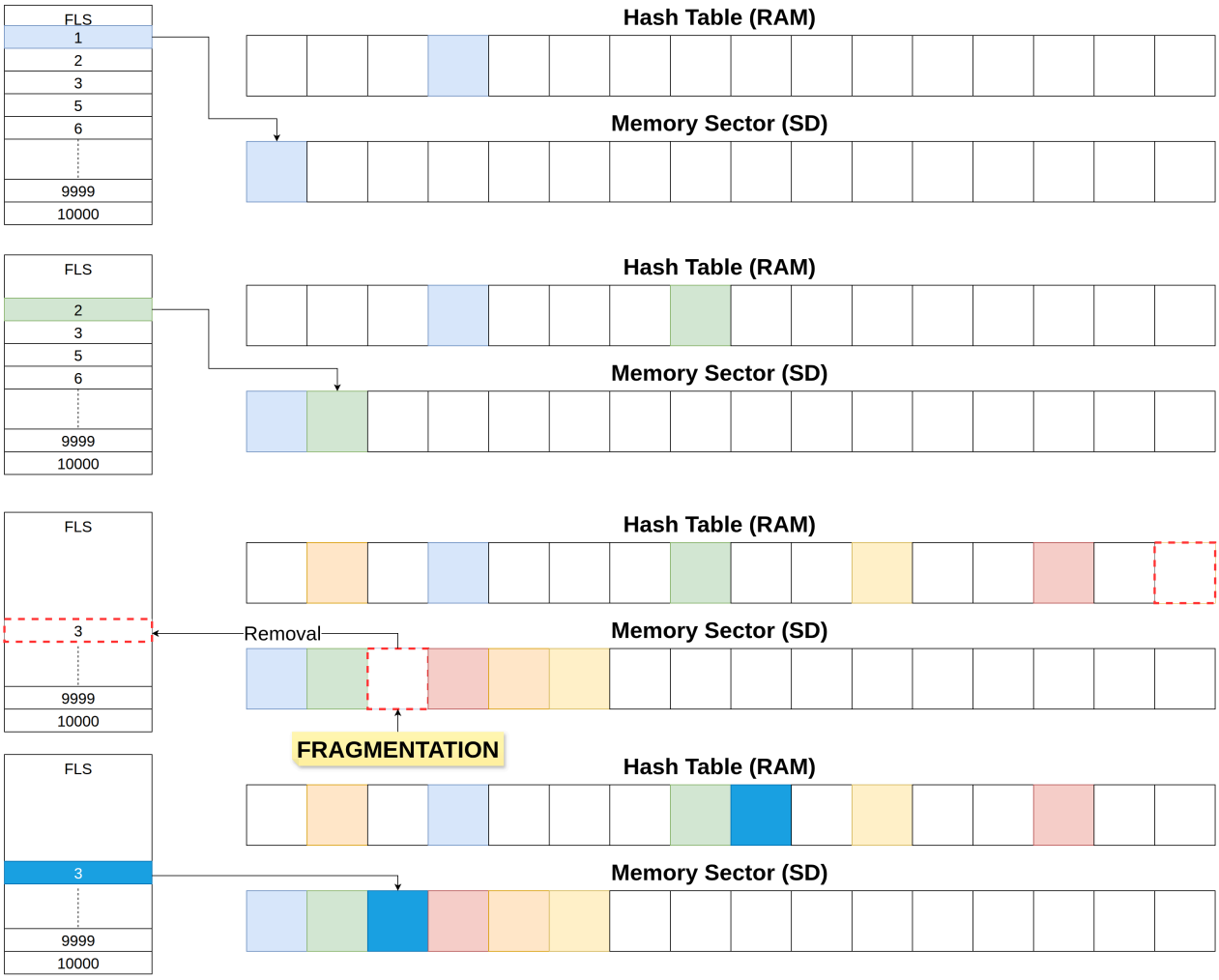



Figure 6: The FLS allocator: `pop` for new sectors, `push` on delete, keeping allocation sequential and memory unfragmented.

# Collision handling and tombstone cleanup

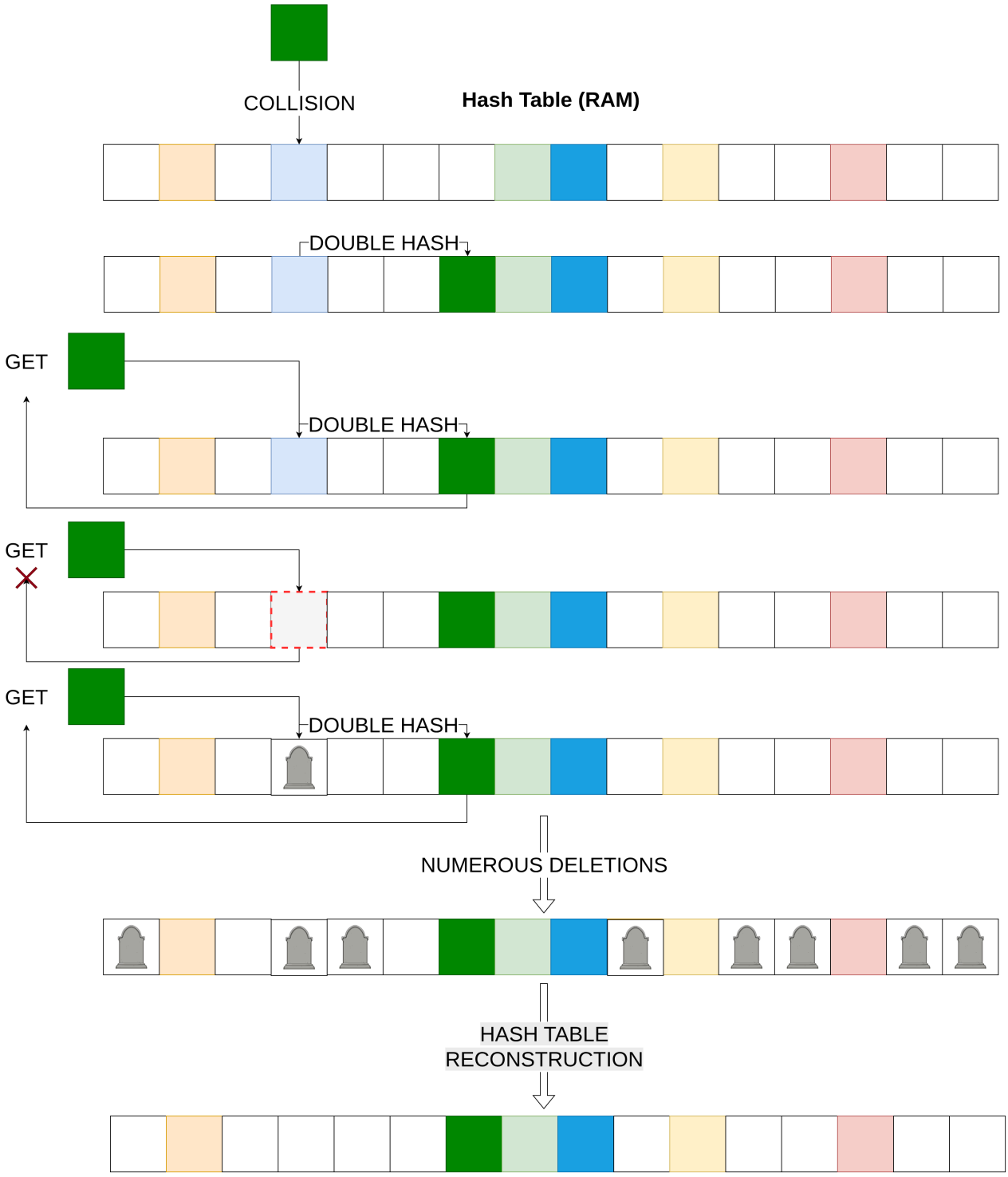



Figure 7: Double hashing with tombstoning. Tombstones are tracked separately from live entries so effective load can be monitored and the table reconstructed if it crosses a threshold – though reconstruction already happens on every power cycle.

For the full narrative and the remaining data-flow diagrams (chat paging, the DB task's request types, phone-number hashing, the poster summary) see `README.md` at the repo root and `diagrams/UniQOS DB Architecture.drawio` (25 pages). The hashing and collision design choices are validated with reproducible measurements in **03 – Benchmarking & Design Evidence**.